# PSAI_010

To submit a proposal, please create a PR against this template in this repo. Please title your file `open-proposal-title.md`, replacing `title` with the name of your project.

# Open Grant Proposal: XDV

**Name of Project:** XDV protocol and node

**Proposal Category:** app-dev

**Proposer:** @molekilla

**Do you agree to open source all work you do on behalf of this RFP and dual-license under MIT and APACHE2 licenses?:** yes

# Project Description

**XDV Protocol** objective is to make any document or metadata, linked data or linked tokens, verifiable within decentralized blockchain stores like Filecoin, decentralized content network like IPFS or hybrid decentralized content network with blockchain features like Swarm Bee. Verifiable Documents can be a verified credential as proposed by the Verifiable Credentials model, an ERC-721 standard NFT (Non Fungible Token), a PKCS#11 or PKCS#12 signed document verifiable with government mandated smart cards or as simple as any binary data signed with a DID that can be authenticated and verified with a proof (eg proof of residence from a KYC Know Your Customer or proof of humanity).

### Previous Work

XDV initially focused on wallets and having those wallets be able to sign documents with government mandated smart cards. A second feature came later where the document was anchored to a blockchain. XDV worked with Swarm V1, which eventually shutdown and went to focus on Swarm Bee. XDV project team moved to IPFS while the new Swarm Bee blockchain was under construction.

One feature requested was to preserve privacy, encryption or zero knowledge technologies were required. After looking for previous work, we stumble upon Ceramic blog and found How to Store Signed and Encrypted Data On IPFS.

We took that paper as base layer for our implementation with a few twists:

- **No server changes**: Customized IPFS implementation are difficult to maintain.
- **Keep it simple**: dag-cbor instead of dag-jose.
- **Delegate missing features to other stacks**: Subscriptions, events and encryption can be done by other complementary APIs.

Another previous work that we manage to create innovation around it is did-jwt. We forked did-jwt and created did-jwt-rsa, which is a DID JWT that signs and verifies RSA Signatures. By accomplishing that, we practically gain DID compatibility with PKCS#11/PKCS#12, which in most governments around the world are required for legal document signing.

And then we asked ourselves, **what if you could do a RSA Signature enabled blockchain**? We found out that there is actually one project, NDID blockchain from Thailand, which is developed with Tendermint.

## Enter XDV: The need for custom tailor made protocol

Initially, Substrate was the top pick, but after much research about market conditions (right now is pretty difficult to find Rust engineers) and current team knowledge, we went with Cosmos SDK and Starport. By June 2021 we explored an initial IPLD Cosmos SDK PoC but after thinking about the infrastructure cost to deploy and operate a blockchain it was decided to keep using Tendermint/Cosmos but with a robust chain platform, for that Secrets Network fits most of the tech use cases and supports WASM using CosmWasm and Rust.

Most of the APIs required to build XDV Protocol are in Go but also in Rust. A first protocol draft must contain the basic primitives, and further protocol versions must be able to build on previous work.

Considering that signature is just one of the many use cases, and that disparate data sources and structures will be more common in the future, XDV Protocol lingua franca is based in IPLD Schemas from Protocol Labs.

Thus, the current XDV Protocol Specification:

### A Layer 2 Solution Approach

L2 is required, as most of the SDKs for these use cases are in Go.

### Primitives

- **Files**: Agnostic CID interface which uses rust-ipld / CosmWasm Smart contracts or go-prime-ipld
- **OffchainDataSources**: An ODBC inspired oracle based connection with BLS Trusted Blockchain Oracle Scheme Based on Aggregate Signature
- More references: https://medium.com/coinmonks/the-simple-price-oracle-verification-based-on-bls-key-aggregation-and-n-of-n-multisignature-902e4aa3c66

### Smart Data Contracts

- **Smart Data Contracts** uses the underlying primitives as API to build next generation ETL / Map Reduce like functions, where every data node is linkable and verifiable.

### HTLC / Atomic Swaps

There are many NFT platform that need metadata decoupling and lack proper way to cross chain swap metadata. Here we'll try to use time lock

contracts to swap metadata uri betweens chains.

# Value

Please describe in more detail why this proposal is valuable for the Filecoin ecosystem. Answer the following questions: - What are the benefits to getting this right? it could replace oracles gen 1, where every data feed requires extensive development time

## Use Cases

#### Cross chain NFT metadata uri token swap

A NFT chain has no EVM presence, but metadata uri is stored in XDV Protocol. They need to be able to peg the ERC-721 compatible token and swap metadata uri while keeping the merkle tree proof / CID in XDV Protocol.

#### Insurance Claims

An `insurance policy` is created with automated legal tech and is connected or integrated with the issuer of the policy. Insurance brokers or directly via website, sells policies to end users. They can get the policy by mail or in digital form. The issuer has decided to use public blockchains to be more transparent, and the broker has decide to use the issuer company REST API to start moving the previous centralized.

To listen and execute next flow, the claim will require additionally integration linked to brokers, issuer (insurance), policy holders agents (VC agent) and other offchain data sources.

- `Claim` is filed thru decentralized automated legal tech (`broker`) and sent to insurance company.
- `Policy` is verified by back office and updated in centralized solution. `Broker` uses XDV Protocol oracle contracts
- After oracle come to a consensus, an event triggers a `smart data contract` and orchestrate trusted data which then is computed and notified to subscribers

#### EDI Orders and Invoicing

XDV protocol integrates EDI orders with offchain workers. The grocery consortium each one has different types of data structures, they are looking for:

- Verify EDI orders signed with RSA
- Convert them to electronic invoice and cloned to XDV Node XDV Storage and configure with RBAC/ACL
- Push invoice to government tax office with government mandated signature using `smart data contracts`

#### COVID-19 verifiable data warehousing

In this use case, you have a pharmaceutical company, government and patients. To keep the diverse data verifiable and able to be query and used for other, non transactional uses, and because cost savings is very important in these implementations, they decide to use XDV Node and implement XDV protocol as following:

- Pharma company has their vaccine data in Ethereum private blockchain, because is private, decentralized chain, it can be thought as a sidechain. These transactions can be duplicated in XDV Node, and keep the data with selective disclosure or encrypted using XDV or IPFS Storage.
- Government keeps its data in CSV Tabular format. They would like to keep these as usual, with XDV Node, because it uses IPLD Schemas, the government selects the codec format for JSON Table. Any request from the data will be render and store as JSON Table. For this to work, they'll need to use oracle which will sync with XDV Node. Because each oracle will use BLS to sign for proof, it should be more secure than other providers.
- Patients get mobile and desktop DApps to keep track of news and events.

Pharma and Government, besides the transactional integration which keeps accountability of vaccine success, can also create `smart data contracts`. A `smart data contract` aggregates, orchestrate and maps linked, verifiable data from the chain transactions and connected.

- What are the risks if you don't get it right? none
- What are the risks that will make executing on this project difficult? finding knowledgable IPLD engineers

# Deliverables

## Metadata / File storage rust-ipld

#### Spec

Create a multiformat `xdv` and migrate PoC to XDV Node (Go) and/or Rust. The API must be 100% compatible with IPFS, ie P0ST `/add` and other relevant REST methods. It must have an interface with events to register `cross chain swaps` events.

It won't be chunk like protocol, more like `microstorage` for usage with NFT, the protocol might have some rate limiting features.

Documentation for client and smart contracts integration for XDV Node L2, Secrets Network, Binance Smart Chain and EVM must be documented with sample code.

## HTLC

#### Spec

`Cross metadata swap` feature involves Flow <> EVM metadata uri swap between tokens.

XDV Node L2 will manage Flow, Ethereum or Swarm Go SDK integrations.

Use CosmWasm Atomic Swap implementation (https://github.com/CosmWasm/cosmwasm-plus/tree/main/contracts/cw20-atomic-swap)

Create an adapter design pattern similar to one used in REN Protocol, and create adapters for Flow and EVM (depending on Secrets Network BSC

bridge API). The adapter interface similar to

```
interface MetadataSwapInterface {
    swap(string from, string to, network toNetwork, string metadatauri)
    Swap(event name, ...args)
    supportsNetwork(type)
}
```

The swap interface depending on the adapter, must be able to do HTLC based on the network `to` argument. This can be verified by calling `supportsNetwork`,

The HTLC events must match any event sequence handle by each token implementation, ie `Mint` event in an BSC NFT token must match tx id in `MetadataSwapInterface` in `Swap` event.

Besides the Flow and EVM implementation, a MockAdapter will be added as sample.

Once tested and QA in testnet, launch in mainnet.

Documentation for client and smart contracts integration for Secrets Network, Binance Smart Chain and EVM must be documented with sample code.

## Compute Data Contracts

### Spec

By using `Secrets Network` vast open source tooling, we can then create a Smart Data Contract or Compute Data Contracts, to emphasis the use of Secrets Network Compute module.

We plan to have a set of smart contracts:

- Oracle contract to get real time data feeds from sources like Splunk, SIEMs, Databases. (https://github.com/enigmampc/secret-oracle/blob/master/src/contract.rs)
- HSM signer contract (https://github.com/enigmampc/secret-vault)

XDV Node L2 API is required here to be able to use the latest official Go SDKs for `go-ipld-prime`, `protobuf`, `go-ethereum`, `ethersphere\bee` and others.

XDV Node L2 must be able to:

- PubSub data feeds from oracles
- REST API to run jobs, create smart data contracts, and any other CRUD operation
- Output IPLD results to smart contract
- ETL interface must accept either IPLD Schemas or Protobuf
- Enable RSA Signing

#### A sample XDV Smart Data Contract might look like

```
        func (xdv XDVDataContract) HelloWorldComputeContract(
            ctx sdk.Context,
            note Note,
        ) NodeAssembler {
            // Inputs

            // 1 - Runtime input
            nodeRuntime, _ := xdv.contracts.Data.ReadDataFromCID(`bafy.............`)

            // 2 - Args input
            nodeArgs, _ := xdv.contracts.Data.ReadDataFromCID(FromWhitelistedArgs(node.myCID))

            // 3 - Oracle data feed
            // XDV oracles MUST translate ANY data feed to multiformat CID to be compatible with Smart Data Contracts
specification
            nodeOracleFeed, _ := xdv.contracts.Data.ReadOracleDataFromCID(oracleID, itemCid)

            // ============================================================
            // Compute or Map  Reduce
            // ============================================================
            np := basicnode.Prototype.Any // Pick a prototype: this is how we decide what implementation will store the in-
data.
            nb := np.NewBuilder()         // Create a builder.
            ma, _ := nb.BeginMap(5)       // Begin assembling a map.
            ma.AssembleKey().AssignString("hey")
            ma.AssembleValue().AssignString("it works!")
            ma.AssembleKey().AssignString("yes")
            ma.AssembleValue().AssignBool(true)

            // Note: pseudocode
            ma.AssembleEntry("runtimelink").AssembleLink(nodeRuntime.Link)
            ma.AssembleEntry("argslink").AssembleLink(nodeArgs.Link)
            ma.AssembleEntry("oraclefeedlink").AssembleLink(nodeOracleFeed.Link)
            ma.Finish()     // Call 'Finish' on the map assembly to let it know no more data is coming.
            n := nb.Build() // Call 'Build' to get the resulting Node.  (It's immutable!)

            // ================================================================================
            // Output store in XDV Protocol smart contracts
```

```
            res, _ := xdv.contracts.Storage.MarshalDagJson(n) // we can add a network/chain type to store it (eg Filecoin)
            // res, _ := xdv.contracts.Storage.MarshalDagCbor(n)

             return res;
            // Render JSON Output:
            // {"hey":"it works!","yes":true, "runtimelink": "bafy.....", argslink: "Qm.......", "oraclefeedlink": "xaadddd
}
        }
```

We should be able to 1) Consume any `structure dataset` (eg CSV, JSON, XML, DB using oracle data feeds) by using multiformats compatible with IPLD and 2) Transform, query and output (also known as map reduce) computed data.

## Development Roadmap

Please break up your development work into a clear set of milestones. This section needs to be very detailed (will vary on the project, but aim for around 2 pages for this section).

For each milestone, please describe: - The software functionality that we can expect after the completion of each milestone. This should be detailed enough that it can be used to ensure that the software meets the specification you outlined in the Deliverables. - How many people will be working on each milestone and their roles - The amount of funding required for each milestone - How much time this milestone will take to achieve (using real dates)

## Primitives

### M1

### Metadata / File storage rust-ipld

- XDV storage (IPLD+Cosmos PoC) - 100%
- XDV Node Provider (CosmJS Client integration with XDV Universal Wallet) - 100%
- Migrate to Secrets Network
- Unit tests
- Documentation
- Integration with XDV BSC Smart Contracts
- MVP 1 Release

Estimation: 6 weeks - 1 Aug - 15 Sept

Take existing PoC, move it to L2 / Rust / Secrets Network, create unit tests and documentation, create sample apps. Then update client package and integrate with existing EVM Smart Contracts

Expect: Metadata compatible with ipld/multiformats Funds: $12 000 Team: Rogelio Morrell

### M2

### HTLC

- Implement HTLC
- Create Adapter / Wrapper chain API
- Implement Flow adapter
- Implement EVM adapter
- Unit tests
- Documentation
- MVP 2 Release

Estimation: 10 weeks - 1 Sept - 1 Dec

Expect: Must be able to swap NFT tokens using M1 metadata feature

Funds: $52 000 Team: Rogelio Morrell, Fernando Romero, Edgar Sucre

### M3

### Compute Data Contracts

- Implement offchain worker
- Implement IPLD / Protobuf Compute Data contracts v1
- Unit tests
- Documentation
- MVP 3 Release

Estimation: 8 weeks - 1 Jan - 1 Mar

Expect: Must be able to compute linked data sources using M1 metadata/files feature

Funds: $40 000 Team: Rogelio Morrell, Fernando Romero, Edgar Sucre

**Note**: Security and cryptographic features built in provided by Secrets Networks

#### Roles

Rogelio Morrell: Lead Architect and Developer Edgar Sucre: Architect and Developer Fernando Romero: Architect and Developer

Total = around 7 months

## Total Budget Requested

RM=7k / month ES=7k / month FR=7k / month

3 deliverables = $104,000

## Maintenance and Upgrade Plans

Yes, we will practically based our business around this open source blockchain and it will be maintain for the foreseeable future.

# Team

## Team Members

### IFESA / IDAO / Fernando Romero

- Rogelio Morrell
- Edgar Sucre
- Fernando Romero

### IFESA Support Team

- Luis Sanchez
- Kendall Kant

## Team Member LinkedIn Profiles

- Rogelio Morrell
- Edgar Sucre
- Fernando Romero

### IFESA Support Team

- Luis Sanchez
- Kendall Kant

## Team Website

- https://testnet.xdv.digital
- https://industriasdao.com
- https://ifesa.tech
- https://docs.xdv.digital

## Relevant Experience

**Rogelio Morrell**, PAID Network Lead Architect, Ikonic.gg NFT Lead Architect, XDV Inventor, created XDV Universal Wallet, XDV Workflow Contracts and `did-jwt-rsa`.

**Edgar Sucre**, Crypto Exchange Alliance, renamed to Fluid Finance, Lead Architect.

**Fernando Romero**, World Class Scala Architect Expert, working with Scala for the last 6 years in Europe.

## Team code repositories

### IFESA Repos

https://github.com/Electronic-Signatures-Industries/

### XDV Node

https://github.com/Electronic-Signatures-Industries/xdv-node

# Additional Information

None